

Modularização e Reutilização de Código em Python

Organizando o Sistema de Biblioteca em funções, módulos e bibliotecas

Prof. Esp Kelrison Coêlho

Objetivo

Compreender como dividir um programa em partes menores, reutilizar funções, importar módulos e utilizar bibliotecas úteis do Python. Ao final, o aluno deverá estar preparado para **refatorar o projeto Sistema de Biblioteca**, deixando o código mais organizado e fácil de manter.

1. Por que modularizar?

Quando um programa cresce, manter tudo dentro de um único arquivo torna o código mais difícil de ler, testar e corrigir. Modularizar significa **dividir o sistema em partes com responsabilidades específicas**.

No Sistema de Biblioteca, por exemplo, podemos separar as operações relacionadas aos livros, aos alunos e aos empréstimos.

Antes	Depois
Um arquivo com todo o código	Arquivos separados por responsabilidade
Trechos repetidos	Funções reutilizáveis
Mais difícil de localizar erros	Código mais organizado

Ideia principal

Um programa modularizado continua sendo um único sistema. A diferença é que seu código passa a ser organizado em funções e arquivos que trabalham em conjunto.

2. Reutilização com funções

Uma **função** reúne instruções que realizam uma tarefa específica. Em vez de repetir o mesmo código várias vezes, criamos a função uma vez e fazemos sua chamada sempre que necessário.

```
def exibir_menu():
    print("1 - Cadastrar livro")
    print("2 - Listar livros")
    print("3 - Sair")

exibir_menu()
```

Também podemos enviar informações para uma função utilizando **parâmetros**:

```
def exibir_livro(titulo, autor):
    print("Título:", titulo)
    print("Autor:", autor)

exibir_livro("Dom Casmurro", "Machado de Assis")
```

E podemos utilizar **return** quando queremos que a função devolva um resultado:

```
def verificar_disponibilidade(quantidade):
    if quantidade > 0:
        return "Disponível"
    else:
        return "Indisponível"

situacao = verificar_disponibilidade(3)
print(situacao)
```

Aplicando ao projeto

Operações que atualmente estão espalhadas pelo Sistema de Biblioteca podem se transformar em funções, por exemplo:

- **cadastrar_livro()**
- **listar_livros()**
- **cadastrar_aluno()**
- **realizar_emprestimo()**
- **exibir_menu()**

Isso prepara o código para o próximo passo: mover funções relacionadas para arquivos separados.

3. Módulos: separando o programa em arquivos

Em Python, um arquivo **.py** pode funcionar como um **módulo**. Assim, funções criadas em um arquivo podem ser importadas e utilizadas em outro.

Uma possível organização do projeto:

```
sistema_biblioteca/  
|-- main.py  
|-- livros.py  
|-- alunos.py  
`-- emprestimos.py
```

Arquivo livros.py:

```
def cadastrar_livro(biblioteca):  
    titulo = input("Título do livro: ")  
    biblioteca.append(titulo)  
    print("Livro cadastrado!")  
  
def listar_livros(biblioteca):  
    for contador in biblioteca:  
        print(contador)
```

Arquivo main.py:

```
from livros import cadastrar_livro, listar_livros  
  
biblioteca = []  
  
cadastrar_livro(biblioteca)  
listar_livros(biblioteca)
```

Outra forma de importar:

```
import livros  
  
biblioteca = []  
livros.cadastrar_livro(biblioteca)  
livros.listar_livros(biblioteca)
```

Observe: o arquivo *main.py* passa a controlar o fluxo principal do sistema, enquanto os demais módulos concentram funções relacionadas a cada responsabilidade.

4. Bibliotecas úteis do Python

Além dos módulos criados por nós, Python possui módulos e bibliotecas prontos. Eles evitam que seja necessário programar novamente funcionalidades comuns.

Data atual com datetime

```
from datetime import date

hoje = date.today()

print("Data do empréstimo:", hoje)
print("Ano:", hoje.year)
print("Mês:", hoje.month)
print("Dia:", hoje.day)
```

No Sistema de Biblioteca, podemos registrar a data em que um empréstimo foi realizado:

```
from datetime import date

def realizar_emprestimo(livro, aluno):
    data_emprestimo = date.today()

    print("Livro:", livro)
    print("Aluno:", aluno)
    print("Data:", data_emprestimo)

realizar_emprestimo("Python Básico", "Ana")
```

Outros exemplos úteis

```
import random

codigo = random.randint(1000, 9999)
print("Código gerado:", codigo)

import math

resultado = math.sqrt(25)
print(resultado)
```

Importante

Neste momento, o objetivo não é decorar bibliotecas. O importante é compreender que **import** permite utilizar funcionalidades existentes em outros módulos, sejam eles criados pela própria equipe ou disponibilizados pelo Python.

5. Exemplo integrado: menu + funções + módulos

Imagine que o projeto já possui o módulo **livros.py**. O arquivo principal pode ficar responsável apenas pelo menu e pela chamada das funções.

livros.py

```
def cadastrar_livro(biblioteca):
    titulo = input("Título: ")
    biblioteca.append(titulo)

def listar_livros(biblioteca):
    if len(biblioteca) == 0:
        print("Nenhum livro cadastrado.")
    else:
        for contador in biblioteca:
            print("-", contador)
```

main.py

```
from livros import cadastrar_livro, listar_livros

biblioteca = []

while True:
    print("\n1 - Cadastrar livro")
    print("2 - Listar livros")
    print("0 - Sair")

    opcao = input("Escolha: ")

    if opcao == "1":
        cadastrar_livro(biblioteca)
    elif opcao == "2":
        listar_livros(biblioteca)
    elif opcao == "0":
        break
    else:
        print("Opção inválida.")
```

Perceba a evolução

Antes: menu, cadastro, listagem e regras no mesmo trecho de código.

Depois: o menu controla o fluxo e chama funções específicas. Essas funções podem ser distribuídas em módulos.

Esse processo de reorganizar um código existente sem alterar o objetivo principal do sistema é chamado de **refatoração**.

6. Exercícios de fixação e próximo passo

Exercício 1 - Transformando em função

Crie uma função chamada **exibir_aluno(nome, matricula)** que receba o nome e a matrícula de um aluno e exiba essas informações na tela.

Exercício 2 - Criando e importando um módulo

Crie o arquivo **alunos.py** com a função **cadastrar_aluno()**. Depois, crie um arquivo **main.py**, importe a função e faça sua chamada.

Exercício 3 - Biblioteca de data

Utilize **from datetime import date** para mostrar a data atual. Em seguida, adapte uma função de empréstimo para exibir também a data em que o empréstimo foi realizado.

Desafio rápido

Observe seu Sistema de Biblioteca atual e identifique pelo menos **quatro trechos** que podem ser transformados em funções. Escreva apenas os nomes que você daria a essas funções.

PRÓXIMO PASSO: REFATORAR O PROJETO COMPLETO

Agora o Sistema de Biblioteca deverá evoluir. Reorganize o projeto utilizando funções e módulos, evitando deixar todas as operações dentro de um único arquivo.

Sugestão de estrutura:

main.py - menu e fluxo principal

livros.py - cadastro, consulta, atualização e remoção de livros

alunos.py - operações relacionadas aos alunos

emprestimos.py - empréstimos e devoluções

Sempre que fizer sentido, reutilize funções e importe recursos prontos, como **datetime**, para registrar datas.

Objetivo da refatoração

O programa deve continuar realizando as operações já desenvolvidas, mas agora com um código mais organizado, legível e reutilizável. A avaliação deve considerar não apenas se o sistema funciona, mas também se o aluno consegue explicar **onde estão as funções, como os módulos se comunicam e por que cada parte foi separada**.